

log4net 2.0.9 + .NET Remoting

Разбор переписки по стенду **Log4NetErrorRepro** · .NET Framework 4.7.2 · 13–16 августа 2026

Короткий вывод

Ошибка «иногда» после `catch + log.Error("Текст", ex)` возникает не из-за разных версий log4net и не из-за поля `long? orderId` на exception. Падает сериализация **CallContext** при формировании remote-ответа, если в `LogicalThreadContext.Properties` лежит **null**. В log4net 2.0.9 метод `ReadOnlyPropertiesDictionary.GetObjectData` вызывает `value.GetType()` без проверки на `null`.

Виноватый стек (верхушка): `Object.GetType()` → `ReadOnlyPropertiesDictionary.GetObjectData` → `BinaryFormatter / remoting`. Чужой пример с `SqlDependency` имел ту же верхушку — к вашей задаче `SqlDependency` не относится.

Стенд

- Клиент и сервер: одна версия **log4net 2.0.9** (как на ПРОМе)
- .NET Remoting TCP `tcp://127.0.0.1:9099/ErrorService`
- Сервер ловит `MyException` и по сценарию логирует / формирует ответ

```
dotnet run --project Log4NetErrorRepro.Server
dotnet run --project Log4NetErrorRepro.Client
```

Сценарии стенда

#	Что делает	Ожидание
1	<code>catch + throw</code> , без лога	клиент получает <code>MyException</code>
2	<code>log.Error("Текст") + throw</code>	то же
3	<code>log.Error("Текст", ex) + throw</code>	то же
4	<code>log.Error("Текст", ex) + DTO</code> без exception	ответ приходит
5	<code>log.Error("Текст", ex) + DTO</code> с <code>MyException</code>	ответ приходит
6	LTC со строками + <code>log.Error</code> + DTO	при одинаковом log4net — OK
7	чтение <code>TargetSite</code> + DTO с exception	ответ приходит
8	<code>LTC[user]=null</code> + <code>log.Error</code> + сериализация <code>CallContext</code>	NRE GetType — как на сервере
9	как 8, затем слот <code>Properties</code> обнуляется через <code>CallContext</code>	может уже не падать (обход)
10	<code>long? orderId=null</code> только на exception, без LTC, return DTO	ответ приходит, <code>orderId</code> не уезжает

Вопрос — ответ

ВОПРОС

Нужен C# стенд net 4.7.2 + log4net 2.0.9, чтобы смоделировать ошибку после `log.Error("Текст", ex)` в `catch(MyException ex)`, когда формируется ответ на remote-вызов.

ОТВЕТ

Собран раствор из трёх проектов: Contracts, Server, Client. Сервер ловит `MyException` и по меню клиента прогоняет варианты лога и remote-ответа.

ВОПРОС

Сценарий 6 падает: не удалось найти сборку log4net 2.0.9. На ПРОМе у клиента и сервера одна версия.

ОТВЕТ

Первая ошибка 6 была артефактом стенда: у клиента log4net не было. На ПРОМе обе стороны с 2.0.9 — эту ошибку «assembly not found» ждать не нужно. Версия закреплена в `Directory.Build.props`.

ВОПРОС

Настоящий сервер падает с верхушкой стека как здесь (трейс чужой, `SqlDependency` не наш):

ОТВЕТ

Совпадает механизм: `BinaryFormatter` сериализует `CallContext`, в нём `log4net.Util.PropertiesDictionary`. В 2.0.9 `GetObjectData` делает `GetType()` на каждом value до проверки на null.

Server stack trace:

```
at System.Object.GetType()
at log4net.Util.ReadOnlyPropertiesDictionary.GetObjectData(...)
at System.Runtime.Serialization.Formatters.Binary.WriteObjectInfo.InitSerialize(...)
at System.Runtime.Serialization.Formatters.Binary.BinaryFormatter.Serialize(...)
... remoting ...
```

ВОПРОС

Можно ли написать `LogicalThreadContext.Properties = null`?

ОТВЕТ

Нет. У `Properties` только getter. Не скомпилируется. Рабочие аналоги:

```
// падает при remote, если value null (сценарий 8)
log4net.LogicalThreadContext.Properties["user"] = null;

// не компилируется
// log4net.LogicalThreadContext.Properties = null;

// аналоги «обнулить Properties»
log4net.LogicalThreadContext.Properties.Clear();
CallContext.LogicalSetData("log4net.Util.LogicalThreadContextProperties", null);
```

ВОПРОС

Может ли `long? OrderId = null` на exception без сериализации поля дать тот же стек?

ОТВЕТ

Само поле на exception — **нет** (сценарий 10: DTO уходит, OrderId просто не сериализуется). Да — только если это null кладут в LogicalThreadContext. Разницы нет:

```
Properties["OrderId"] = null;
Properties["OrderId"] = ex.OrderId; // тоже null, это тот же сценарий 8
```

ВОПРОС

На ПРОМе ошибка «иногда», когда попадаем в catch.

ОТВЕТ

Ломается не сам catch и не log.Error. В этот момент в LTC уже бывает null (user / OrderId / requestId пустые не в каждом запросе), а потом сериализуется CallContext вместе с remote-ответом. Локально поля обычно заполнены — поэтому «у нас работает».

Как выловить виноватый property на сервере

В catch **до** формирования remote-ответа:

```

catch (MyException ex)
{
    log.Error("Текст", ex);

    object raw = CallContext.LogicalGetData("log4net.Util.LogicalThreadContextProperties");
    IDictionary dict = raw as IDictionary;
    if (dict == null)
    {
        log.Warn("LTC: слот пуст");
    }
    else
    {
        foreach (DictionaryEntry entry in dict)
        {
            if (entry.Value == null)
                log.Error("LTC GUILTY null key=" + entry.Key);
            else
                log.Info("LTC ok key=" + entry.Key + " type=" + entry.Value.GetType().FullName);
        }
    }
    // дальше remote-ответ
}

```

Ищите в логе LTC GUILTY null key=... . Нужны `using System.Collections;` и `using System.Runtime.Remoting.Messaging;` .

Почему падает log4net 2.0.9

Упрощённо GetObjectData (проверка null — **после** GetType):

```

foreach (DictionaryEntry entry in InnerHashtable)
{
    string key = entry.Key as string;
    object value = entry.Value;
    bool serializable = value.GetType().IsSerializable; // NRE, если value == null
    if (key != null && value != null && serializable)
        info.AddValue(key, value);
}

```

Обход перед remoting:

`CallContext.FreeNamedDataSlot("log4net.Util.LogicalThreadContextProperties");` либо не писать в LogicalThreadContext значения, которые бывают null (`if (ex.OrderId != null)`), либо `ThreadContext` вместо Logical (он не уходит в CallContext).

Что не является этой ошибкой

- Разные версии log4net на клиенте/сервере (на ПРОМе они одинаковые)
- SqlDependency (чужой пример с той же верхушкой стека)
- `long? OrderId` на exception без GetObjectData — поле просто не уедет
- `Properties = null` — так написать нельзя

